

データマイニング特論

第13回

深層学習の概観と使いどころ

【修正】スライド28,29,30

本科目シラバスの確認 3（授業計画）

#	日程	内容
第 1 回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第 2 回	4/23（木）	pandasによるデータ操作の基礎
第 3 回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第 4 回	5/7（木）	データの前処理と特徴量エンジニアリング
第 5 回	5/14（木）	分類の基礎 ― k近傍法と決定木
第 6 回	5/21（木）	アンサンブル学習
第 7 回	5/28（木）	線形分類モデルと回帰
第 8 回	6/4（木）	モデル評価・選択と不均衡データ
第 9 回	6/11（木）	クラスタリング手法
第 10 回	6/18（木）	アソシエーション分析と異常検知
第 11 回	6/25（木）	テキストマイニング
第 12 回	7/2（木）	時系列データマイニング
第 13 回	7/9（木）	深層学習の概観と使いどころ
第 14 回	7/16（木）	公平性・倫理と再現性
第 15 回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1



深層学習の仕組み

ニューロン・多層・学習の流れ・表現学習

深層学習は、もう身近にある

【でも今日の問いは『で、自分はいつ使うべき？』】



対話AI

ChatGPT・生成AI



画像・動画生成

テキストから画像を作る



自動運転

カメラ映像の認識



科学研究

AlphaFold（構造予測）

華やかな成功例は多い。

しかし本当に大事なのは『万能ではない道具を、いつ選ぶか』を判断できること。

復習：線形モデル（第7回）

線形モデルは「入力の重み付き和」

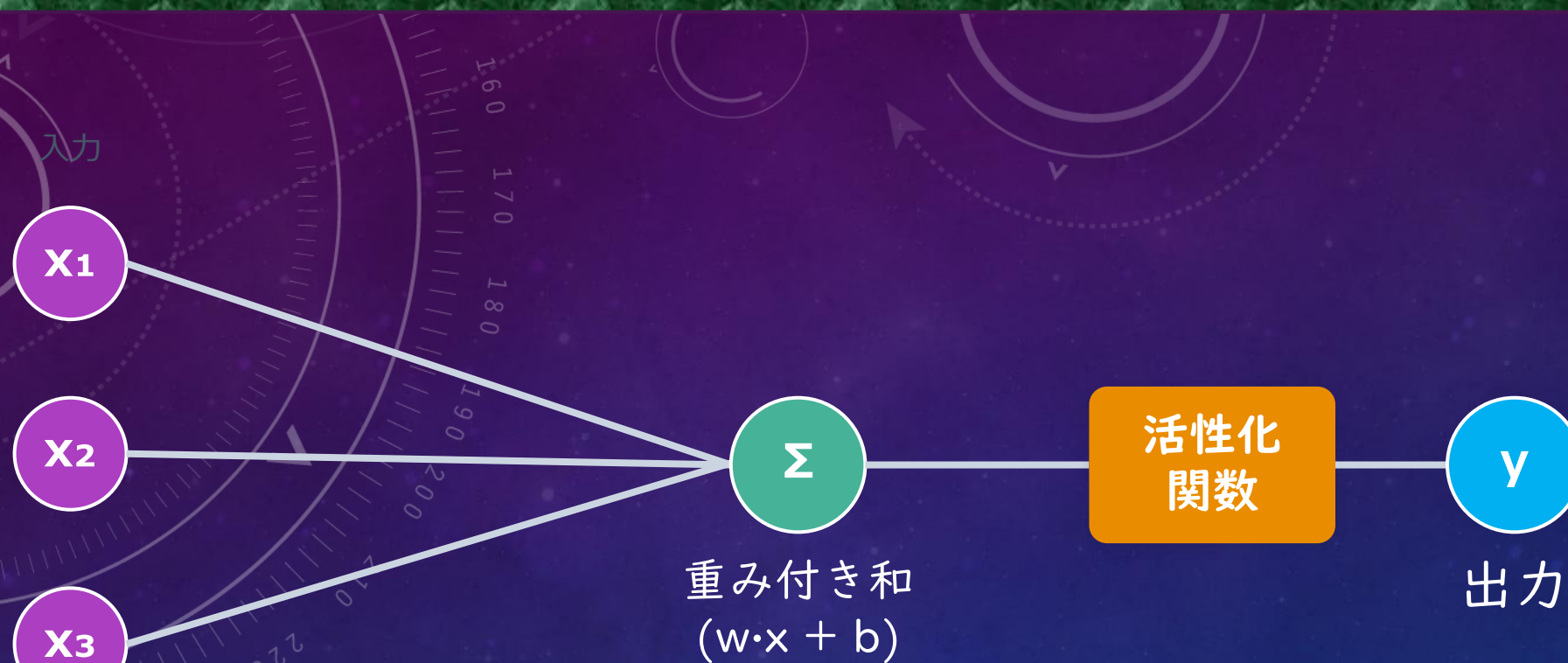
$$y = w_1x_1 + w_2x_2 + \dots + b \quad (\text{重み } w \text{ と切片 } b \text{ を学習})$$

長所：速い・解釈しやすい

短所：直線的な関係しか表せない

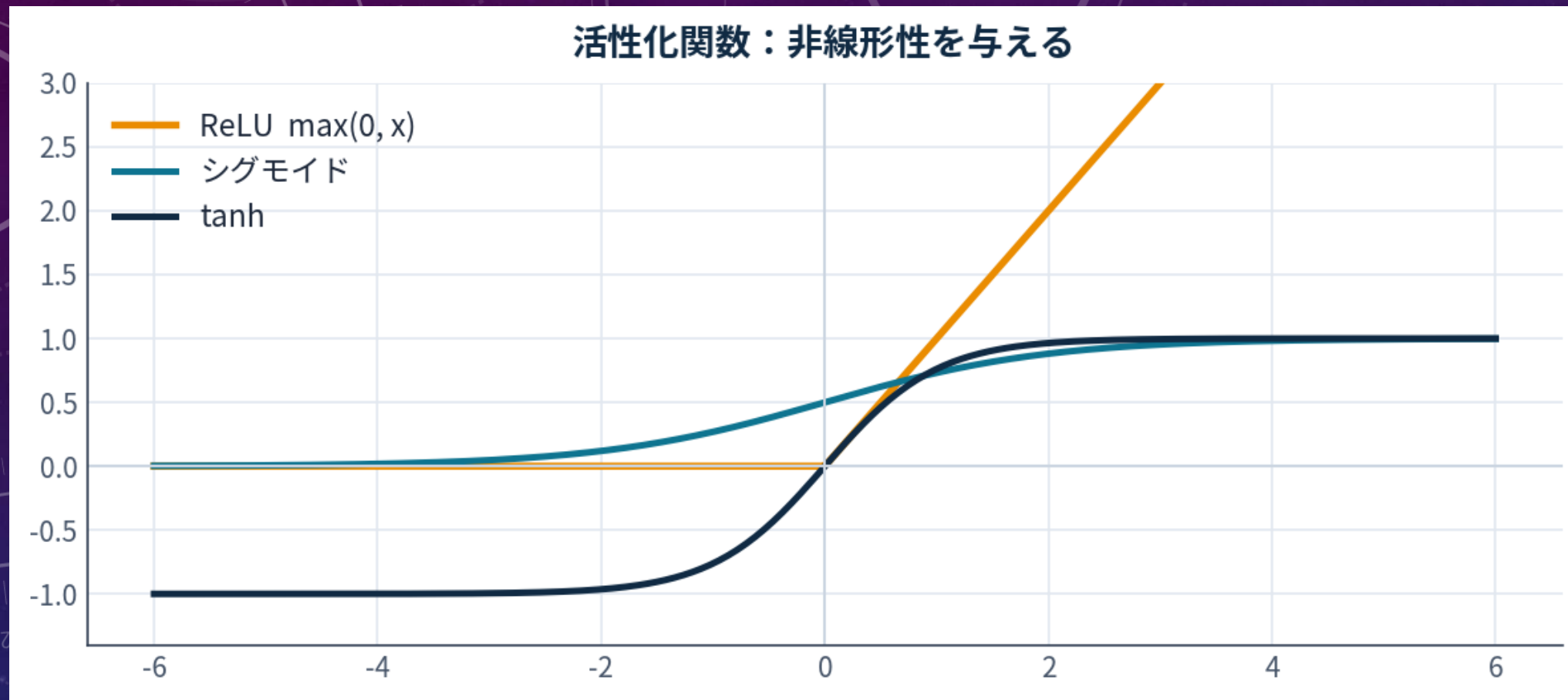
この「重み付き和」が、実は深層学習のいちばん小さな部品になる。

ニューロン = 線形結合 + 活性化関数



ニューロン1個は、第7回のロジスティック回帰とほぼ同じ。
違いは、これを大量に並べて重ねること。

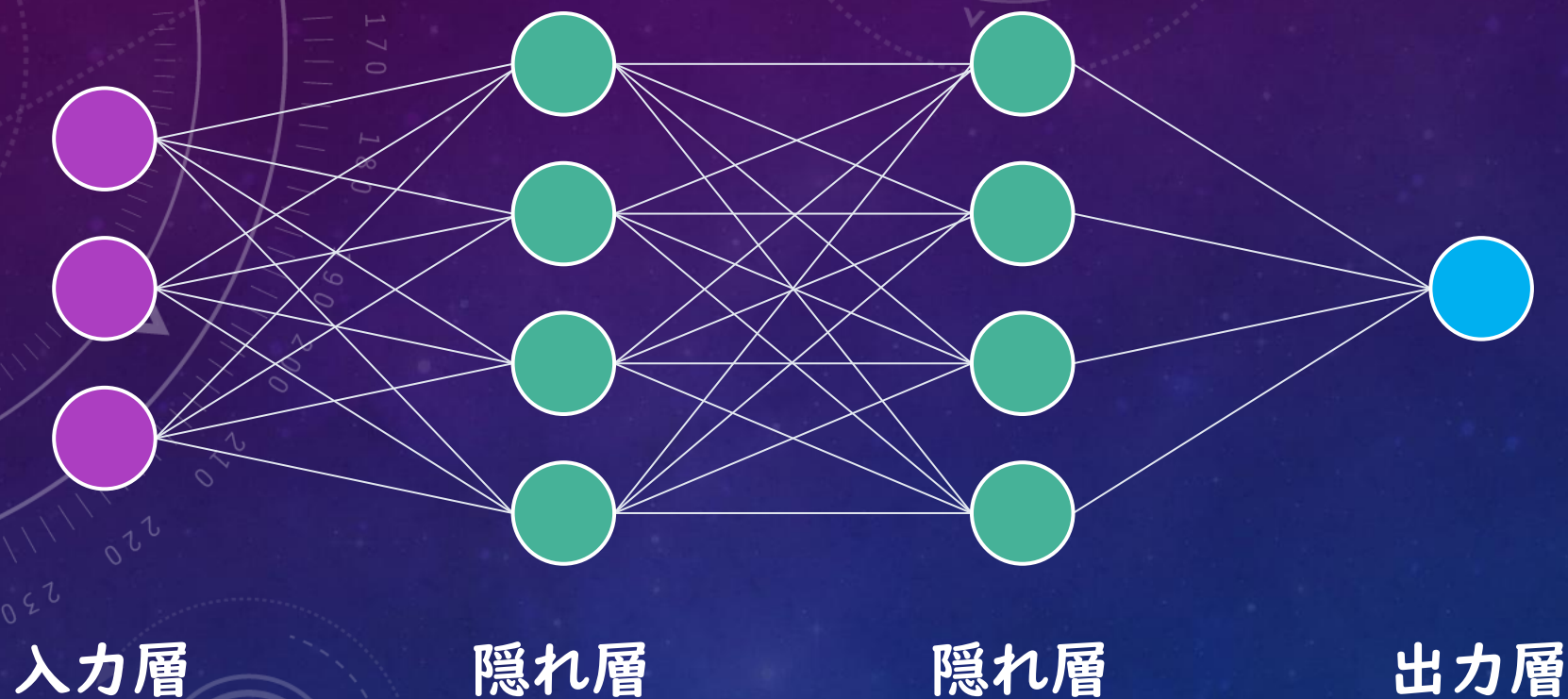
活性化関数はなぜ必要か



活性化関数がないと、何層重ねても結局ただの線形モデルに戻ってしまう。
非線形性を与えることで、複雑な関係を表せる。今はReLUが標準。

多層化 = ニューラルネットワーク

【隠れ層を積み重ねる = 深層（ディープ）】



層を深くするほど複雑な関数を表現できる。
これが「深層」学習の名前の由来。

学習の直感：間違いを測って、少し直す



① 損失を測る

予測と正解の
ズレ（損失）を計算



② 勾配を求める

損失を減らす方向を
計算（誤差逆伝播）



③ 重みを直す

その方向へ少しだけ
更新（勾配降下）

これを何万回もくり返して、ズレが小さくなるように重みを自動調整する。
数式の詳細は今日は踏み込まない。

何が新しいのか = 表現学習

【深層学習の核心：特徴量を『自動で』学ぶ】

従来（第4回）

- 人間が特徴量を設計する
- ドメイン知識が要る
- 手間がかかる
- 例：面積・築年数・立地を手で作る

深層学習

- 生データから特徴を自動で学ぶ
- 層が進むほど抽象的な特徴に
- 画像の画素→輪郭→部品→顔
- 設計不要（その代わり大量データが要る）

特徴量エンジニアリングを機械に任せられる。
これが画像や言語で深層学習が強い理由。

2



代表的なアーキテクチャ

表／画像／系列／言語

データの種類の、使う道具が変わる



表データ

MLP / 木系



画像

CNN



系列・時系列

RNN・LSTM

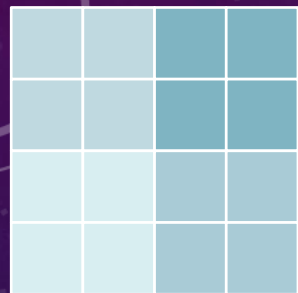


言語・テキスト

Transformer

「深層学習」と一括りにせず、データの形に合ったアーキテクチャを選ぶ。
以降、代表を順に見る。

CNN = 画像のための道具



画像（画素）



畳み込み
フィルタ



特徴マップ
(輪郭・模様)



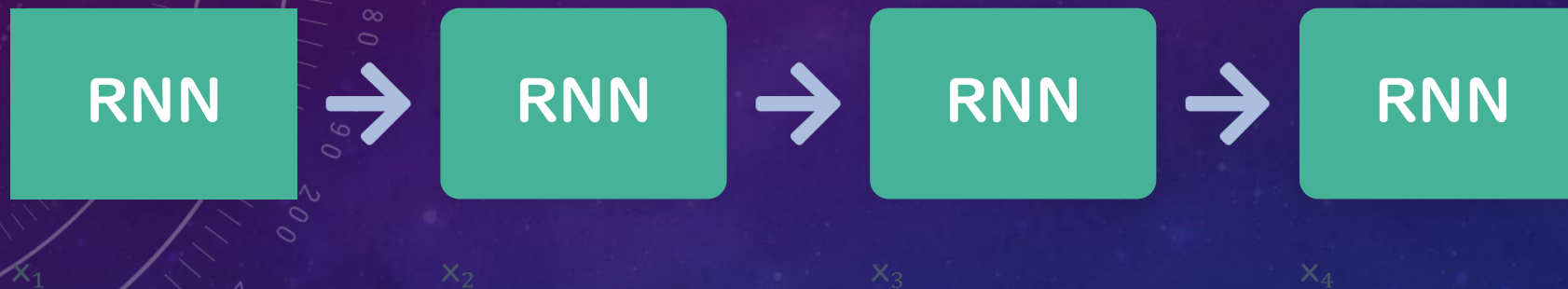
分類

畳み込み（コンボリューション） = 小さなフィルタで画像全体をなぞり、局所パターンを検出。

「近くの画素は関係が深い」という画像の性質を活かす。
層が深いほど、輪郭→部品→物体と抽象化。

RNN・LSTM = 系列のための道具

状態を次に渡す →



前の時点の情報（状態）を次に引き継ぎながら、系列を順番に処理する。
第12回の時系列データがまさに対象。長い系列の記憶を改善したのが LSTM。

Transformer・大規模言語モデル = 言語のための道具



文中のどの単語がどの単語と関係するか（注意機構）を学ぶ。
離れた語どうしの関係も一度に捉えられ、並列計算にも向く。

第11回との接続

BoWやTF-IDFの次に来る、言語の主役。文の「意味」をベクトルで扱う。

ChatGPTの正体

巨大なTransformerを大量の文章で事前学習したもの。次の単語を予測する仕組み。

『大規模言語モデル（LLM）』は、
この Transformer を極端に大きく・大量データで学習したもの。

転移学習：ゼロから学習しない

【現代の常識 — 学習済みモデルを使い回す】

学習済みモデル
(巨大データで
事前学習済み)



再利用

自分の課題
(少ないデータで
微調整するだけ)



高精度

少ないデータ・少ない計算でも高い精度が出せる。

「巨大モデルを自分でゼロから訓練する」ことは普通しない。
これが重要な使いどころ。

3

深層学習の『使いどころ』

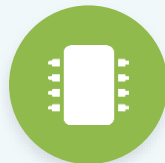
いつ使い、いつ使わないかを見極める



深層学習のコスト（ただではない）



大量のデータが
要る



GPU など
計算資源



ブラックボックス
（説明が難しい）



チューニングが
難しい



過学習しやすい

これらのコストに見合うだけの効果があるか？
それを判断するのが「使いどころ」。

表データでは、木系（GBDT）が今も強い

【深層学習は万能ではない（第6回アンサンブルの回収）】



表形式データでは、勾配ブースティング木が深層学習に匹敵、
しばしば凌駕する。

研究でも繰り返し確認されている（例：Grinsztajn et al., 2022, NeurIPS）。

なぜか：表データは特徴量ごとに意味・尺度がばらばらで、
しきい値的な判断が多い。木はこれを自然に扱える。
DLは前処理と大量データを要求する。

使いどころ判断チェックリスト

- 1 データの種類は？** 画像・音声・言語・系列 → DL が有利
表 → まず木系
- 2 データ量は十分か？** 数万～数百万件あるか（少なければ DL は不利）
- 3 計算資源はあるか？** GPU・時間・運用コストを払えるか
- 4 解釈性は必要か？** 説明責任が要るなら DL は不利（第14回へ）
- 5 強いベースラインに勝てるか？** まず単純な手法と必ず比較する

研究での使いどころ

【データの種類が DL 向きなら、強力な武器になる】



構造・配列

AlphaFold
(タンパク質構造予測)



画像・動画

顕微鏡像の分類、
行動の自動解析



音声・系列

鳴き声や信号の
分類・時系列モデル

みなさんの研究でも、画像・音声・配列のように『DL向きのデータ』なら検討の価値がある。表形式の統計解析はこの限りでない。

まとめ：深層学習は『道具の一つ』

- 1 データの種類で選ぶ** 画像・音声・言語・系列＝DL有利／表＝まず木系
- 2 表データではDLは万能でない** 強いベースライン（GBDT）と必ず比較
- 3 表現学習が核心** 特徴量を自動で学ぶのがDLの強み
- 4 転移学習が現代の常識** ゼロから訓練せず学習済みを使い回す
- 5 コストと効果を天秤に** データ量・計算・解釈性を見て判断

The background is a dark blue gradient with a starry texture. On the left side, there are several concentric circular patterns. One large circle has a degree scale from 140 to 260. Other smaller circles have arrows indicating clockwise or counter-clockwise rotation. The text '本日の演習' is centered in the middle of the image.

本日の演習

本日の環境準備

いろいろ試してみよう！

```
# 日本語対応のモジュールをインストールする(その1)
!pip install japanize-matplotlib
# 日本語対応のモジュールをインストールする(その2)
!apt-get install -y fonts-ipafont-gothic
```

```
# 各ライブラリーをインポートする
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
try:
    import japanize_matplotlib # noqa: F401
except Exception:
    pass
plt.rcParams["axes.unicode_minus"] = False

RNG = np.random.default_rng(20260709)
```


データの読み込み (1)

いろいろ試してみよう！

データ読み込み

```
LISTINGS_URL = ("https://data.insideairbnb.com/japan/kant%C5%8D/tokyo/"
                "2023-06-29/visualisations/listings.csv")
FEAT_NUM = ["latitude", "longitude", "minimum_nights", "number_of_reviews",
            "reviews_per_month", "availability_365", "calculated_host_listings_count"]
FEAT_CAT = ["neighbourhood", "room_type"]

def load_listings():
    try:
        df = pd.read_csv(LISTINGS_URL)
        df["price"] = (df["price"].astype(str).str.replace(r"¥$,", "", regex=True)
                      .replace("", np.nan).astype(float))
        df = df[FEAT_NUM + FEAT_CAT + ["price"]].dropna(subset=["price", "room_type", "neighbourhood"])
        df["reviews_per_month"] = df["reviews_per_month"].fillna(0)
        df = df[(df["price"] > 0) & (df["price"] < df["price"].quantile(0.99))]
        print(f"[実データ] listings を読み込み: {len(df):,} 件")
        return df.reset_index(drop=True), True
    except Exception as e:
        print(f"[フォールバック] 実データを取得できません({type(e).__name__})。合成データで続行。")
        return synthesize_listings(), False
```

#次に続く。。。

データの読み込み (2)

いろいろ試してみよう！

```
def synthesize_listings(n=15000):
    wards = ["Shinjuku", "Shibuya", "Taito", "Sumida", "Toshima", "Nakano", "Setagaya", "Ota", "Katsushika", "Arakawa"]
    wp = dict(zip(wards, [1.0, 1.05, 0.85, 0.75, 0.7, 0.6, 0.55, 0.45, 0.35, 0.4]))
    w = np.array([2.2, 2.0, 1.6, 1.3, 1.1, 0.9, 0.9, 0.8, 0.7, 0.7]); w = w/w.sum()
    ward = RNG.choice(wards, n, p=w)
    rooms = ["Entire home/apt", "Private room", "Hotel room", "Shared room"]
    re_ = {"Entire home/apt": 1.0, "Private room": 0.45, "Hotel room": 0.7, "Shared room": 0.25}
    room = RNG.choice(rooms, n, p=[0.62, 0.30, 0.05, 0.03])
    mn = np.clip(RNG.lognormal(0.7, 0.8, n).astype(int), 1, 365)
    nrev = RNG.poisson(np.clip(RNG.lognormal(2.2, 1.1, n), 0, None)).astype(int)
    rpm = np.round(np.clip(RNG.gamma(1.5, 0.7, n), 0, None), 2)
    av = np.clip((RNG.beta(1.4, 1.6, n)*365).astype(int), 0, 365)
    hl = np.clip(RNG.lognormal(0.6, 1.0, n).astype(int), 1, None)
    lat = 35.68 + RNG.normal(0, 0.045, n); lon = 139.75 + RNG.normal(0, 0.055, n)
    wpar = np.array([wp[x] for x in ward]); ent = (room=="Entire home/apt"); shr = (room=="Shared room")
    # 木が有利になる「交互作用・非単調・しきい値」を含む価格モデル
    price = (3600*np.array([re_[r] for r in room])*(0.6+0.8*wpar)
              *(1+0.9*ent*(wpar>0.8))
              *(0.75+0.5*np.exp(-((av-180)**2)/(2*70**2)))
              *np.where(mn<=3, 1.15, np.where(mn>=30, 0.8, 1.0))
              *(1+0.5*(nrev>20)*ent-0.3*(nrev>20)*shr)
              *(1+RNG.normal(0, 0.25, n)))
    # 交互作用: 一棟貸し×中心区
    # 非単調: 稼働率の山
    # しきい値: 最低宿泊数
    # 交互作用: レビュー×部屋
```

#次に続く。。。

データの読み込み (3)

いろいろ試してみよう！

```
price = np.clip(price,1500,None).astype(int)
return pd.DataFrame({"neighbourhood":ward,"room_type":room,"latitude":lat.round(5),
    "longitude":lon.round(5),"minimum_nights":mn,"number_of_reviews":nrev,
    "reviews_per_month":rpm,"availability_365":av,
    "calculated_host_listings_count":hl,"price":price})
```

```
df, IS_REAL = load_listings()
```

目的変数:高価格帯か(中央値以上の二値分類)。`price` は特徴量から除外。

```
thr = int(df["price"].median())
```

```
y = (df["price"] >= thr).astype(int).values
```

```
X = df[FEAT_NUM + FEAT_CAT].copy()
```

```
print(f"目的変数=高価格帯か(中央値 {thr}) 正例率 {y.mean():.3f} 特徴量 {X.shape[1]}個\n")
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.base import clone
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.ensemble import HistGradientBoostingClassifier
```

```
from sklearn.metrics import accuracy_score, roc_auc_score
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
```

#次に続く。。

データの読み込み (4)

いろいろ試してみよう！

前処理の部品

```
pre_scaled = ColumnTransformer([("num", StandardScaler(), FEAT_NUM),  
                                ("cat", OneHotEncoder(handle_unknown="ignore"), FEAT_CAT)])  
pre_ord = ColumnTransformer([("cat", OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1), FEAT_CAT)],  
                             remainder="passthrough") # 先頭2列=カテゴリ, 残り=数値  
  
def make_logreg(): return Pipeline([("pre", pre_scaled), ("clf", LogisticRegression(max_iter=1000))])  
def make_mlp_raw(): return Pipeline([("pre", pre_ord), ("clf", MLPClassifier((64,32), max_iter=200,  
random_state=0))])  
def make_mlp_ok(hidden=(64,32), max_iter=300):  
    return Pipeline([("pre", pre_scaled), ("clf", MLPClassifier(hidden, max_iter=max_iter, random_state=0))])  
def make_gbdt(): return Pipeline([("pre", pre_ord), ("clf", HistGradientBoostingClassifier(categorical_features=[0,1],  
random_state=0))])  
  
def evaluate(name, pipe, Xa=Xtr, ya=ytr):  
    t = time.time(); pipe.fit(Xa, ya); ft = time.time()-t  
    pr = pipe.predict_proba(Xte)[: , 1]; p = pipe.predict(Xte)  
    acc = accuracy_score(yte, p); auc = roc_auc_score(yte, pr)  
    print(f"{name:30s} 正解率={acc:.3f} AUC={auc:.3f} 学習時間={ft:5.2f}s")  
    return {"name": name, "acc": acc, "auc": auc, "fit": ft}  
def run(name, pipe):  
    t = time.time(); pipe.fit(Xtr, ytr); ft = time.time() - t  
    p = pipe.predict(Xte); pr = pipe.predict_proba(Xte)[: , 1]  
    acc = accuracy_score(yte, p); auc = roc_auc_score(yte, pr)  
    print(f"{name:32s} 正解率={acc:.3f} AUC={auc:.3f} 学習時間={ft:5.2f}s")  
    return {"name": name, "acc": acc, "auc": auc, "fit": ft}
```

#次に続く。。

データの読み込み (5)

いろいろ試してみよう！

```
results = []
print("=== 比較 ===")

# ステップ1:MLP(スケーリングなし) — NN特有の落とし穴を体感
pre_raw = ColumnTransformer(
    [("cat", OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1), FEAT_CAT)],
    remainder="passthrough")
mlp_raw = Pipeline([("pre", pre_raw),
                    ("clf", MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=200, random_state=0))])
results.append(run("MLP(スケーリングなし)", mlp_raw))

# ステップ2:MLP(標準化+One-Hot) — 正しい前処理
pre_scaled = ColumnTransformer([
    ("num", StandardScaler(), FEAT_NUM),
    ("cat", OneHotEncoder(handle_unknown="ignore"), FEAT_CAT)])
mlp_ok = Pipeline([("pre", pre_scaled),
                   ("clf", MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=300, random_state=0))])
results.append(run("MLP(標準化+One-Hot)", mlp_ok))

# ステップ3:勾配ブースティング(最小前処理・カテゴリはネイティブ対応)
pre_gb = ColumnTransformer(
    [("cat", OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1), FEAT_CAT)],
    remainder="passthrough")
gb = Pipeline([("pre", pre_gb),
               ("clf", HistGradientBoostingClassifier(categorical_features=[0, 1], random_state=0))])
results.append(run("勾配ブースティング(最小前処理)", gb))
```

データの読み込み（6）

[実データ] listings を読み込み: 11,055 件

目的変数=高価格帯か（中央値 14286） 正例率 0.502 特徴量 9個

=== 比較 ===

MLP（スケーリングなし）

正解率=0.583 AUC=0.654 学習時間= 5.63s

/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached
warnings.warn(

MLP（標準化+One-Hot）

正解率=0.757 AUC=0.831 学習時間=42.16s

勾配ブースティング（最小前処理）

正解率=0.783 AUC=0.859 学習時間= 0.76s

可視化 (1)

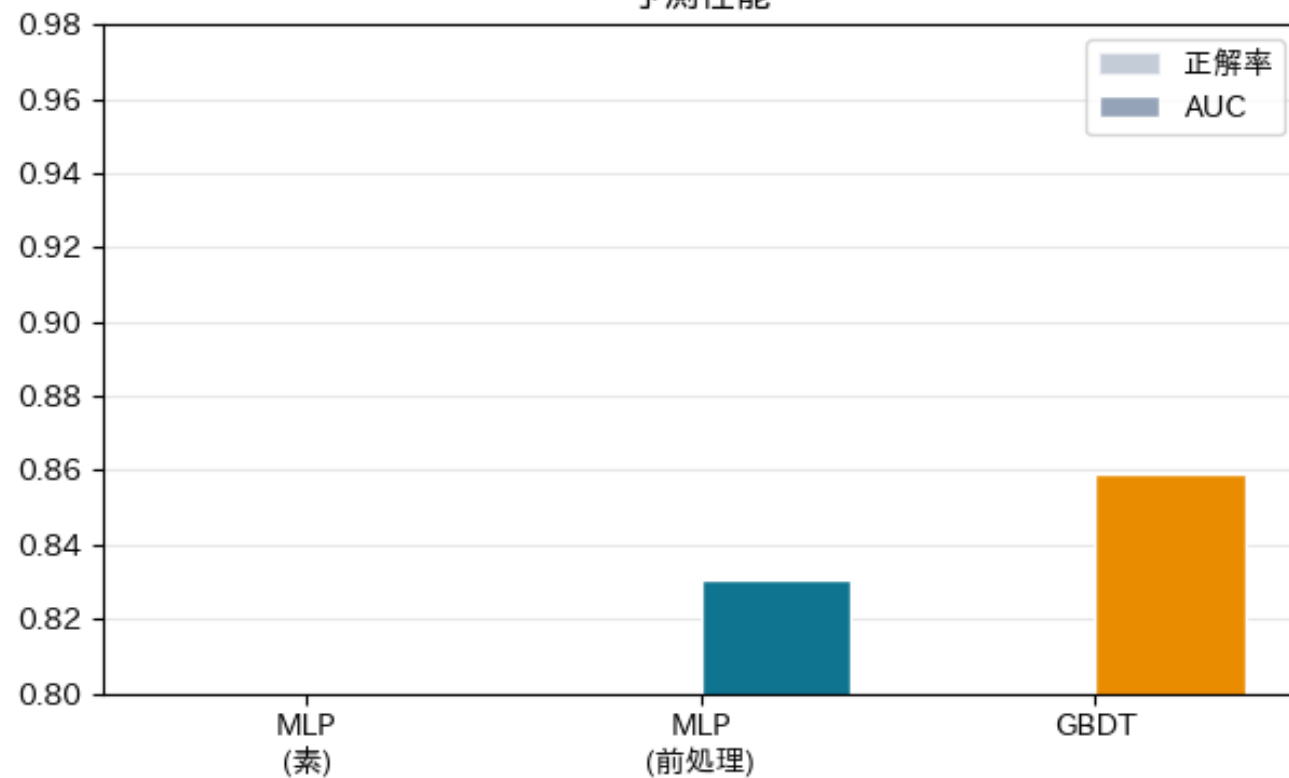
いろいろ試してみよう！

```
labels = ["MLP×n(素)", "MLP×n(前処理)", "GBDT"]
acc = [r["acc"] for r in results]; auc = [r["auc"] for r in results]; fit = [r["fit"] for r in results]
colors = ["#94A3B8", "#0E7490", "#EA8C00"]

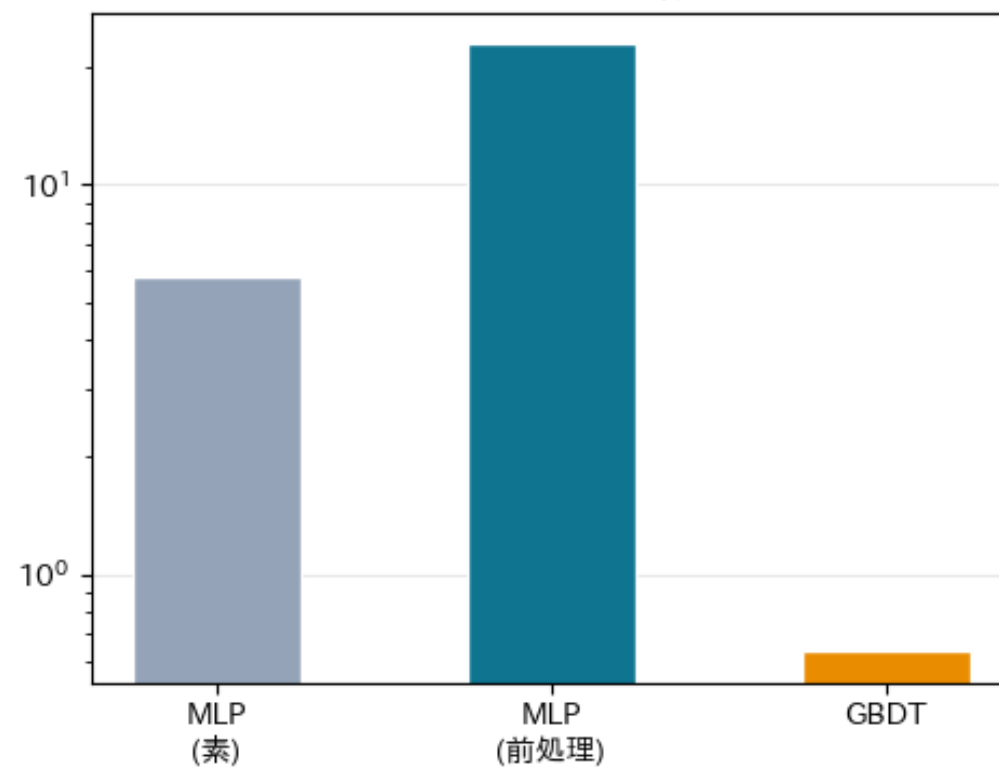
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4), gridspec_kw={"width_ratios": [1.3, 1]})
x = np.arange(3); w = 0.38
a1.bar(x - w/2, acc, w, label="正解率", color=colors, alpha=0.55, edgecolor="white")
a1.bar(x + w/2, auc, w, label="AUC", color=colors, edgecolor="white")
a1.set_xticks(x); a1.set_xticklabels(labels); a1.set_ylim(0.8, 0.98)
a1.set_title("予測性能"); a1.legend(); a1.grid(axis="y", alpha=.3); a1.set_axisbelow(True)
a2.bar(x, fit, 0.5, color=colors, edgecolor="white"); a2.set_yscale("log")
a2.set_xticks(x); a2.set_xticklabels(labels); a2.set_title("学習時間(秒・対数)")
a2.grid(axis="y", alpha=.3); a2.set_axisbelow(True)
plt.tight_layout(); plt.show()
```

可視化 (2)

予測性能

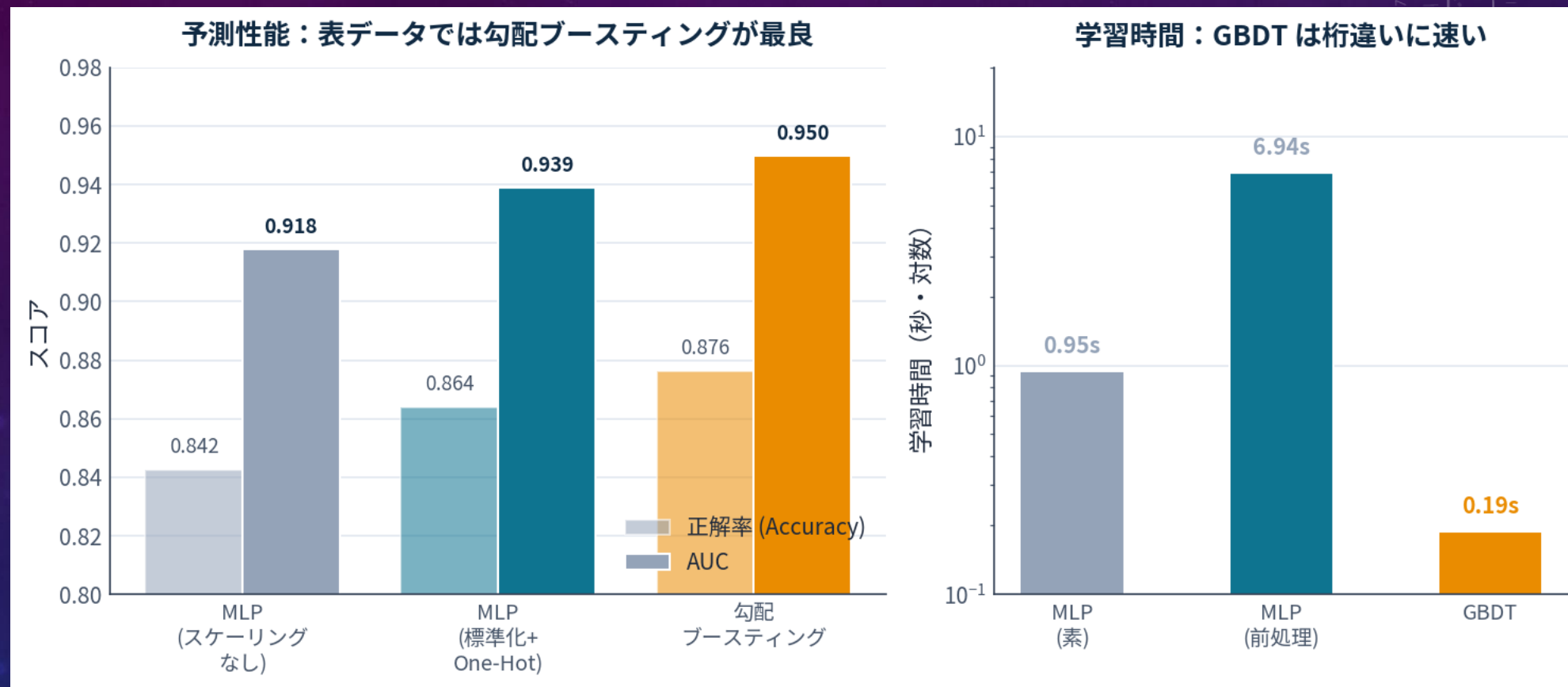


学習時間 (秒・対数)



可視化 (3)

【Airbnb 相当データ 15,000件・実行結果】



AUC 0.950 (GBDT) > 0.939 (MLP)、学習は約37倍速く、前処理も最小。

さらに MLP はスケーリングなしだと精度が落ちる (AUC 0.918→0.939) = NN 特有の落とし穴。

【基本】4モデルの正面比較（Ⅰ）

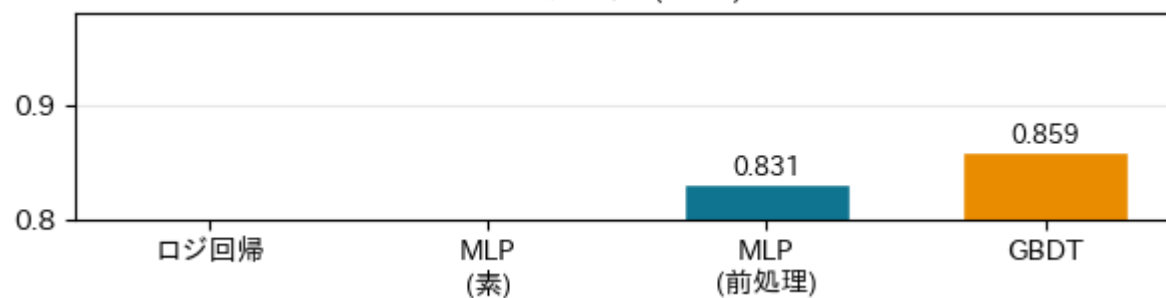
```
res = []
res.append(evaluate("ロジスティック回帰(第7回)", make_logreg()))
res.append(evaluate("MLP(スケーリングなし)", make_mlp_raw()))
res.append(evaluate("MLP(標準化+One-Hot)", make_mlp_ok()))
res.append(evaluate("勾配ブースティング(GBDT)", make_gbdtdt()))

labels = ["ロジ回帰", "MLP¥n(素)", "MLP¥n(前処理)", "GBDT"]
colors = ["#64748B", "#94A3B8", "#0E7490", "#EA8C00"]
auc = [r["auc"] for r in res]; fit = [r["fit"] for r in res]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4), gridspec_kw={"width_ratios":[1.3,1]})
a1.bar(range(4), auc, 0.6, color=colors, edgecolor="white")
for i, u in enumerate(auc): a1.text(i, u+0.003, f"{u:.3f}", ha="center", va="bottom", fontsize=9)
a1.set_xticks(range(4)); a1.set_xticklabels(labels); a1.set_ylim(0.8, 0.98)
a1.set_title("予測性能 (AUC)"); a1.grid(axis="y", alpha=.3); a1.set_axisbelow(True)
a2.bar(range(4), fit, 0.6, color=colors, edgecolor="white"); a2.set_yscale("log")
a2.set_xticks(range(4)); a2.set_xticklabels(labels); a2.set_title("学習時間 (秒・対数)")
a2.grid(axis="y", alpha=.3); a2.set_axisbelow(True)
plt.tight_layout(); plt.show()
```

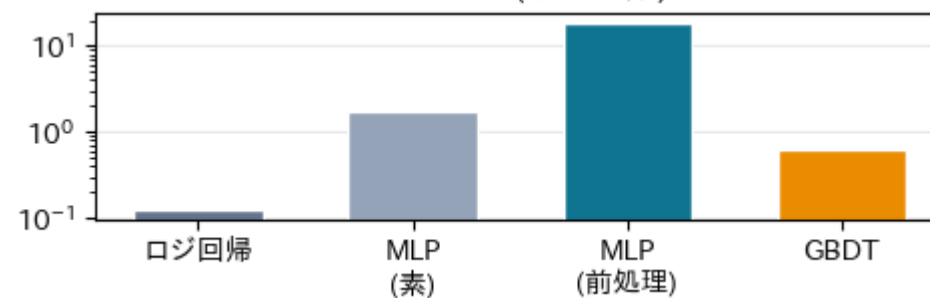
【基本】4モデルの正面比較（2）

ロジスティック回帰(第7回)	正解率=0.607	AUC=0.658	学習時間= 0.13s
MLP (スケーリングなし)	正解率=0.583	AUC=0.654	学習時間= 1.75s
<code>/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300)</code>			
<code>warnings.warn(</code>			
MLP (標準化+One-Hot)	正解率=0.757	AUC=0.831	学習時間=18.64s
勾配ブースティング(GBDT)	正解率=0.783	AUC=0.859	学習時間= 0.62s

予測性能 (AUC)



学習時間 (秒・対数)



【発展Ⅰ】データ量を変えると？ — 学習曲線（Ⅰ）

いろいろ試してみよう！

```

sizes = [500, 1000, 2000, 4000, 8000, len(Xtr)]
auc_mlp, auc_gb = [], []
for nsub in sizes:
    idx = RNG.choice(len(Xtr), size=min(nsub, len(Xtr)), replace=False)
    Xs, ys = Xtr.iloc[idx], ytr[idx]
    m = clone(make_mlp_ok()); m.fit(Xs, ys)
    g = clone(make_gbdm()); g.fit(Xs, ys)
    am = roc_auc_score(yte, m.predict_proba(Xte)[: , 1])
    ag = roc_auc_score(yte, g.predict_proba(Xte)[: , 1])
    auc_mlp.append(am); auc_gb.append(ag)
    print(f" n={nsub:>5}: MLP AUC={am:.3f} GBDT AUC={ag:.3f}")

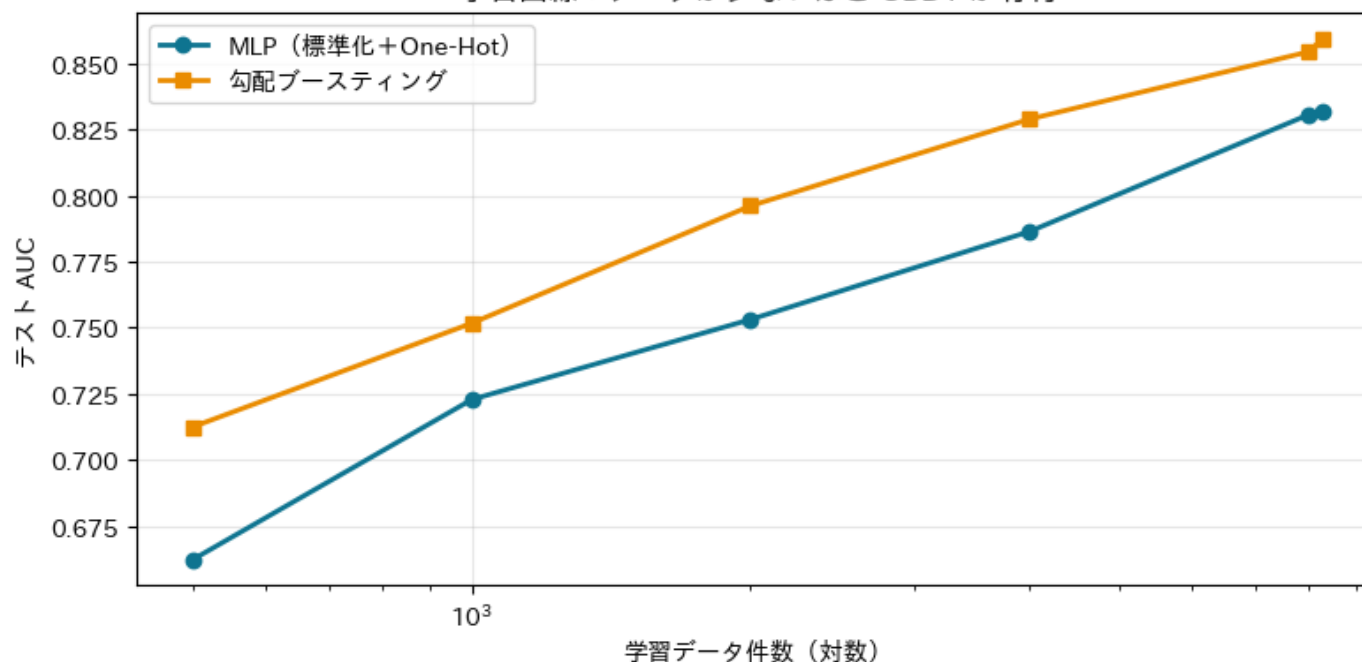
plt.figure(figsize=(8, 4.2))
plt.plot(sizes, auc_mlp, "o-", color="#0E7490", lw=2, label="MLP(標準化+One-Hot)")
plt.plot(sizes, auc_gb, "s-", color="#EA8C00", lw=2, label="勾配ブースティング")
plt.xscale("log"); plt.xlabel("学習データ件数(対数)"); plt.ylabel("テスト AUC")
plt.title("学習曲線:データが少ないほど GBDT が有利"); plt.legend(); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()

```


【発展1】データ量を変えると？ — 学習曲線（2）

```
n= 500: MLP AUC=0.662  GBDT AUC=0.712
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
warnings.warn(
n= 1000: MLP AUC=0.723  GBDT AUC=0.752
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
warnings.warn(
n= 2000: MLP AUC=0.753  GBDT AUC=0.796
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
warnings.warn(
n= 4000: MLP AUC=0.786  GBDT AUC=0.829
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
warnings.warn(
n= 8000: MLP AUC=0.831  GBDT AUC=0.854
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
warnings.warn(
n= 8291: MLP AUC=0.832  GBDT AUC=0.859
```

学習曲線：データが少ないほど GBDT が有利



【発展2】MLP を大きくすると？ — 構造の探索（1）

いろいろ試してみよう！

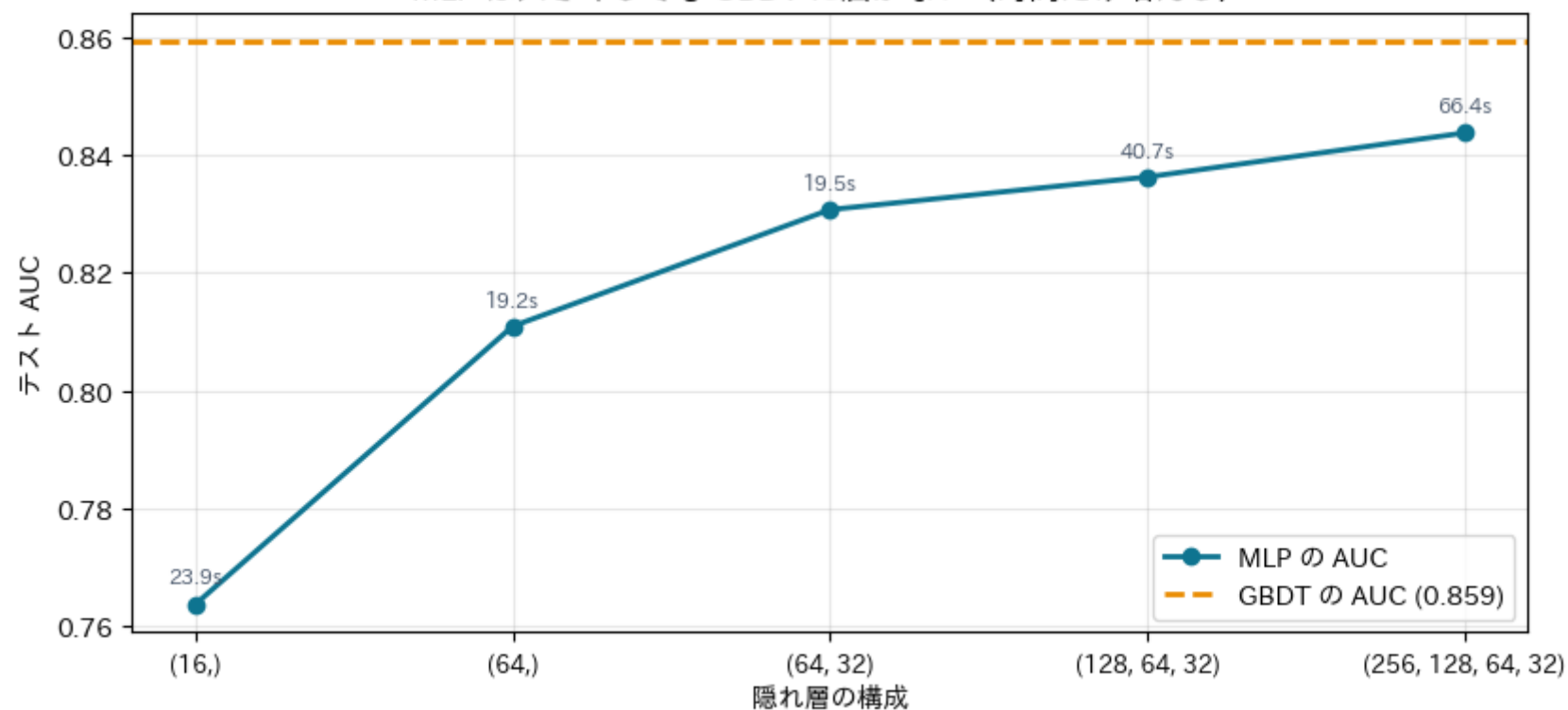
```
archs = [(16,), (64,), (64, 32), (128, 64, 32), (256, 128, 64, 32)]
auc_a, fit_a = [], []
for h in archs:
    m = make_mlp_ok(hidden=h, max_iter=300)
    t = time.time(); m.fit(Xtr, ytr); ft = time.time()-t
    a = roc_auc_score(yte, m.predict_proba(Xte)[:, 1])
    auc_a.append(a); fit_a.append(ft)
    print(f" 隠れ層 {str(h):20s}  AUC={a:.3f}  学習時間={ft:5.2f}s")
gbdt_auc = res[3]["auc"]

fig, ax = plt.subplots(figsize=(8.5, 4.2))
xs = range(len(archs))
ax.plot(xs, auc_a, "o-", color="#0E7490", lw=2, label="MLP の AUC")
ax.axhline(gbdt_auc, color="#EA8C00", ls="--", lw=2, label=f"GBDT の AUC ({gbdt_auc:.3f})")
for i, (a, f) in enumerate(zip(auc_a, fit_a)):
    ax.annotate(f"{f:.1f}s", (i, a), textcoords="offset points", xytext=(0, 8), ha="center", fontsize=8,
color="#475569")
ax.set_xticks(list(xs)); ax.set_xticklabels([str(h) for h in archs], fontsize=9)
ax.set_xlabel("隠れ層の構成"); ax.set_ylabel("テスト AUC")
ax.set_title("MLP は大きくしても GBDT に届かない(時間だけ増える)")
ax.legend(); ax.grid(alpha=.3)
plt.tight_layout(); plt.show()
```

【発展2】MLP を大きくすると？ — 構造の探索（2）

隠れ層 (16,)	AUC=0.764	学習時間=23.88s
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic		
warnings.warn(		
隠れ層 (64,)	AUC=0.811	学習時間=19.24s
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic		
warnings.warn(		
隠れ層 (64, 32)	AUC=0.831	学習時間=19.54s
隠れ層 (128, 64, 32)	AUC=0.836	学習時間=40.65s
隠れ層 (256, 128, 64, 32)	AUC=0.844	学習時間=66.42s

MLP は大きくしても GBDT に届かない（時間だけ増える）



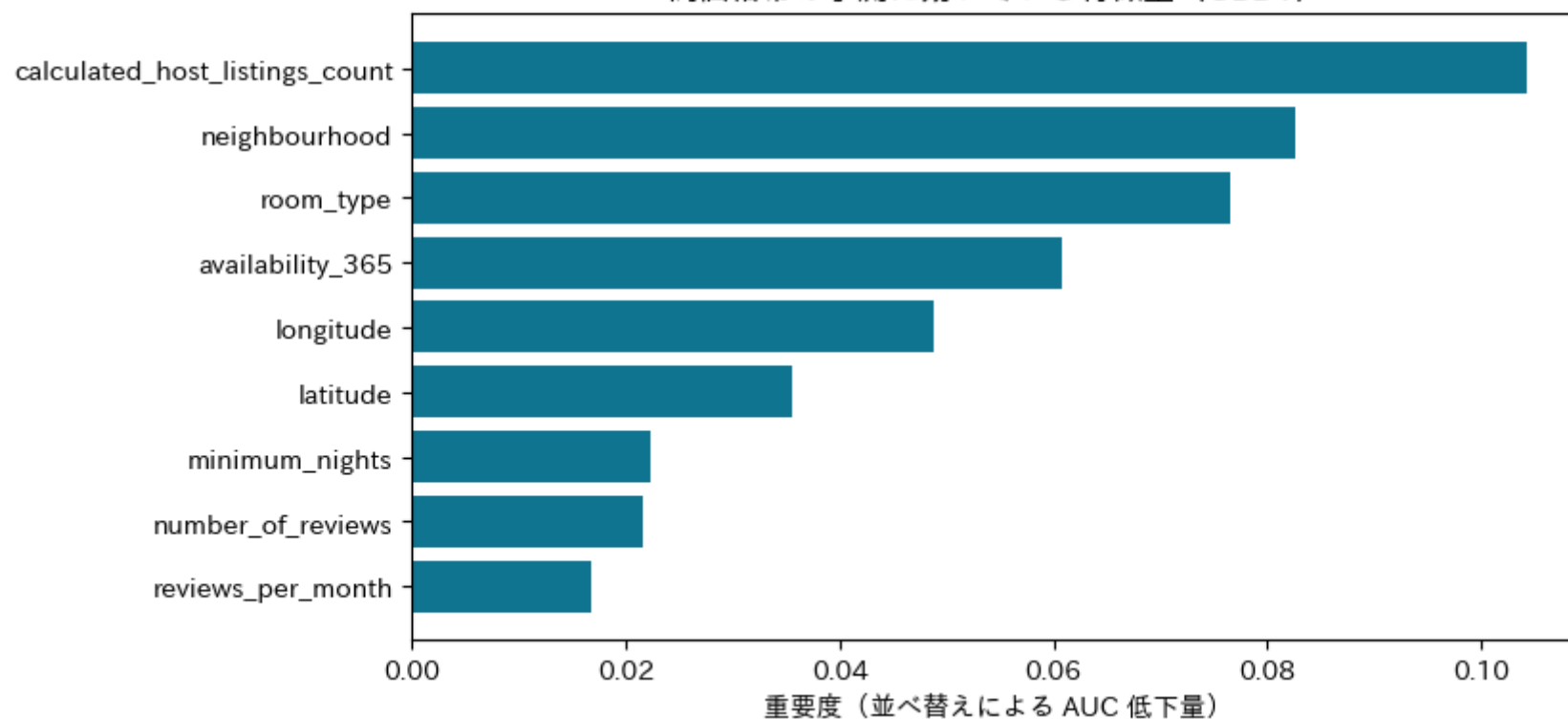
```
from sklearn.inspection import permutation_importance
gb = make_gbdtd(); gb.fit(Xtr, ytr)
imp = permutation_importance(gb, Xte, yte, n_repeats=5, random_state=0, scoring="roc_auc")
order = np.argsort(imp.importances_mean)
feat_names = list(X.columns)
for i in order[::-1]:
    print(f" {feat_names[i]:32s} 重要度={imp.importances_mean[i]:.3f}")

plt.figure(figsize=(8.5, 4.2))
plt.barh([feat_names[i] for i in order], imp.importances_mean[order], color="#0E7490")
plt.xlabel("重要度(並べ替えによる AUC 低下量)")
plt.title("高価格帯の予測に効いている特徴量(GBDT)")
plt.tight_layout(); plt.show()
```


【発展3】どの特徴量が効いている？ — 特徴量重要度（並べ替え法）（2）

calculated_host_listings_count	重要度=0.104
neighbourhood	重要度=0.083
room_type	重要度=0.077
availability_365	重要度=0.061
longitude	重要度=0.049
latitude	重要度=0.036
minimum_nights	重要度=0.022
number_of_reviews	重要度=0.022
reviews_per_month	重要度=0.017

高価格帯の予測に効いている特徴量（GBDT）



4

本日の課題

頑張ってください。



課題

【課題】 期限：2026/7/15（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第13回（7/9）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力
!jupyter nbconvert --to html "aaa.ipynb"

【ADSセミナー】

本学の「先端データ科学研究センター」が主催するセミナーが6/25（木）から4週にわたって開催されます。

内容が盛りだくさんなので、**研究室を選択する前の参考になる**かと思います。

事前登録フォームから申し込みあり。

【Point】 普段の授業では聞けない話もあるかも！？

ADSセミナー

対象 東京情報大学
全学部・学年/大学院生/教職員

会場 1号館 201教室

【第1回】 6/25（木） 16:30 - 18:00

講師： 釣井 達也 先生
総合情報学科 准教授（数理情報研究ユニット）



量子ウォークに潜む群構造について

量子ウォークはランダムウォークの量子版と解釈される数学モデルであり、その振る舞いはランダムウォークと大きく異なります。本セミナーでは、量子ウォークの簡単な具体例を通してランダムウォークとの違いを観察します。特に、量子ウォークに潜む群構造に着目し、この群構造を活用して、量子ウォークの特徴的な挙動のひとつである「周期」を求める方法について紹介します。

【第2回】 7/2（木） 16:30 - 18:00

講師： 早稲田 篤志 先生
総合情報学科 准教授（情報セキュリティ研究ユニット）



個人情報保護と機械学習

プライバシーに関する情報は解析して使用することで有益な情報を得られることが期待できます。一方、漏洩等で外部に漏れると大きな問題となる場合があります。そこで、基本的なプライバシー保護の指標を紹介し、機械学習との関係を考察します。

【第3回】 7/9（木） 16:30 - 18:00

講師： 西村 明 先生
総合情報学科 教授（情報メディア学系）



音響法科学（オーディオ・フォレンジクス）

音響法科学は、音響学の原理と技術を、犯罪捜査・法廷および不正等の検知のために活かす研究と学問を指します。話者識別、雑音除去と音声強調、録音属性推定に大別され、本講演では、前二者を概観したのち、AI音声の判別を含む後者の研究について概説します。

【第4回】 7/16（木） 16:30 - 18:00

講師： 小早川 睦貴 先生
総合情報学科 准教授（生命情報研究ユニット）



身体情報から読みとる心の働き

心理学的研究で心を読み取る際には、さまざまな人間の「反応」を測ることが不可欠です。一般的には反応時間や正答率、表情など目に見える反応を使いますが、発汗や視線など、目に見えない反応などを使って心を読む心理学的研究を紹介します。

<参加登録>

右のQRコードを読み取り、参加登録フォームへアクセスの上、必要事項を記入してお申し込みください。



受付期間： 5/26（火）～ 7/16（木）

<https://forms.office.com/r/JRmJe3qNmh>

早期研究体験プロジェクト
（対象：学部1・2年次）

本プログラムに興味のある学生は、先端データ科学研究センターまでお問い合わせください。

随時
募集中

<お問い合わせ>

東京情報大学 総合情報学部
先端データ科学研究センター
ads@rsch.tuis.ac.jp

